

Weight — Tool Build (mechanics)

A calculator with one number key. A number costs as many taps as its value. Everyday Tools — Fordham Wong

Live link	fwong29.github.io/reusable-studio-engine/everyday-tools/
Repository	github.com/fwong29/reusable-studio-engine — project in <code>everyday-tools/</code>
Commit history	/commits/main — build spans 2026-09-18 and 2026-09-22; break log below is keyed to these hashes
Recording	submitted as a second file on this assignment (core loop narrated at the live URL, ending on <code>git log</code>)

BREAK LOG

Each entry is a moment the tool broke past its simplest case, and the commit that fixed it. Every hash exists in `git log`.

1. `e0c7ea0` — 2026-09-18 — **the failure the README named didn't exist**. The README claimed dividing by zero was the one failure the system could produce. Pushing on it: entering a number requires at least one tap, so a `0` divisor can never be committed — the `Error` branch is unreachable through the UI. Corrected the README to say so and kept the guard as defensive code, rather than advertising a failure that can't happen.
2. `07f859c` — 2026-09-18 — **the accent stopped meaning "effort" at scale**. The intent reserves terracotta to mean effort. Watching a tally climb past ~20, the unbroken row of pills read as a warning bar, not a count. Grouped the marks in fives, so a long count reads as rhythmic clusters and the color keeps its single meaning at any length.
3. `c440c88` — 2026-09-22 — **a number after = errored**. Reproduce: `2 + 2 = 4`, then one tap showed `4 1` (two numbers, no operator), and `=` then threw `Error`. A tap after a result now starts a fresh calculation; an operator after a result still continues from it.
4. `3fd6e95` — 2026-09-22 — **the seam lied above 40**. Reproduce: 45 taps showed `45` on the numeral but only 40 marks, so the seam under-reported the real count with no signal. Past the 40-mark cap the panel now appends a `+N more` indicator, keeping the seam honest at any count.

BACK-END ARCHITECTURE

What data does this tool need?

Three values: the current tally (taps toward the number being entered), the committed expression (a list of numbers and operators), and a flag for whether the current tally has been touched since the last commit. Nothing about who is using it or what the number means.

Where is it stored?

In four JavaScript variables in `main.js` (`currentTally`, `tokens`, `tallyTouched`, `justEvaluated`). Browser memory only.

Is it temporary or persistent?

Temporary. A page reload erases everything. No `localStorage`, no cookie, no database.

Does the system need memory between sessions?

No — and it must not have any. The Dignity Clause says the tool records nothing; the Refusal Clause says the friction never changes with use. Both are only true if the tool forgets. Statelessness is the design, not a shortcut.

Does the system require AI inference?

No. The rule is fixed and mechanical: one tap adds one to the tally. There is nothing to infer.

How many API calls are realistically required?

Zero. The page is three static files. The only network request is a Google Fonts stylesheet, and the page works without it.

What happens if the API fails?

There is no API to fail. The one arithmetic edge, division by zero, is guarded and would render `Error` in the accent color — but as break-log entry 1 found, the entry model makes it unreachable. The webfont is the only external request: it carries `display=swap`, and the tally marks are CSS elements rather than glyphs, so on a slow or failed load the meaning-carrying seam renders identically and only the numerals briefly fall back to system monospace.

LAYERS

LAYER	WHERE	WHAT
Input	<code>index.html</code> buttons → <code>click</code> listeners	Tally key, four operators, <code>=</code> , <code>C</code> , <code>⌫</code>
Logic	<code>pressTally</code> , <code>pressOperator</code> , <code>pressEquals</code> , <code>evaluate</code> ...	Increment the tally; commit it as a number on an operator or <code>=</code> ; evaluate with precedence
Output	<code>render</code> → <code>#display</code> , <code>#tallyCount</code> , <code>#tallyMarks</code>	Running expression, tally numeral, one mark per tap (grouped in fives)

CHOICE POINT

The tally key is the choice point. A normal keypad makes the decision for you — one tap, digit entered. Here the decision is spread across every tap: each one is a small, visible act of continuing. The tally marks under the numeral are the seam — they show exactly how far you've committed, so the key never looks broken while a number is still being built.

BEHAVIOR INTEGRITY CHECK

- **Interrupts where the intent claimed?** Yes — during the behavior, at the moment a quantity becomes a number. Nothing before, nothing after.
- **Avoids shame, surveillance, manipulation?** Stores nothing, knows nothing about what the number means, never comments on it. The cost of entering 7 is identical for every person, every time.
- **Designs friction or exploits it?** The friction is proportional to the number and to nothing else — no escalation, no personalization, no reward.
- **Minimal?** One number key, four operators, three controls. The decimal point was removed because it doesn't fit a tally. Nothing else was added.

HOW THE BUILD WAS MADE

The two 09-22 fixes are the shape of the process: a case pushed past the simple path, a break found in hand, a single change, a message that names it. The break log ties each to its hash; the recording (submitted alongside) narrates the core loop and ends on `git log`.